29 July 2026

CSE 203 Code a7z4xque

DSA (Data structure algorithm

CT 4 টা (array থেকে)

CT 1 week 4/5 দিবে (আগে পরে 20 মার্ক)
(searching sorting সহিত)

4 টা CT এর Best 3

Mid এর আগে CT 2

Linked list শেষ হওয়ার পর CT 3

Tree, graph ,, ,, CT 4

Final এ মিলে সব আসবে

Introducton to algorithm

THOMAS H. CORMEN

Data s... ...schaum series.pdf

LEC-1 Web view তে open করলে প্রথমের কয়টা formatting Powerpoint এ open করা লাগে

LEC-1

Page 1-11

30 July 2026

Slide 12

Linear search

Set k:=1 and LOC:=0

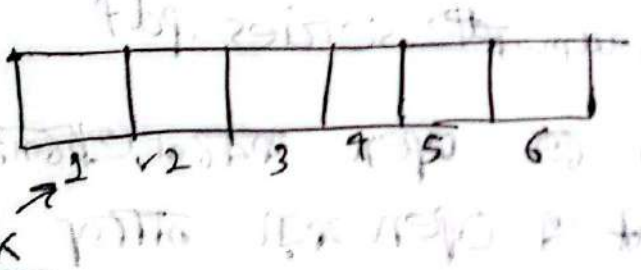
Repeat steps 3 and 4 while k<=N

If Key = L[k] then set LOC:=k, print: Loc and exit

Set k:=k+1

If Loc = 0 the Write: Item is not in list

Exit



location = 0

Binary Search

~~Binary Search~~

1	2	3	4	5	6	7
6	10	20	30	55	75	190

1. key = 55

Beg = 1
End = 7

R_1 (Round 1)

$$Mid = \frac{Beg + End}{2} = \frac{1+7}{2} = 4$$

L[Mid] < Key
L[4] < 55
↓ ↓
30

→ Beg = Mid + 1 = 4 + 1 = 5

R_2

Beg = 5
End = 7
Mid = 6

L[Mid] > Key
L[6] > 55
↓
75

End = Mid − 1 = 6 − 1 = 5

R_3

Beg = 5
End = 5
Mid = 5

L[Mid] = key
L[5] = 55
↓ ↓
55 55

LOC = Mid

~~Bubble sort~~ (Ascending)
Binary search

1. Set LOC := 0, Beg := 1, End := N and
   Mid := (Beg + End) / 2
2. Repeat steps 3 to 6 while Beg <= End
3. If Key < L[Mid], then set End := Mid - 1
4. Else if key > L[Mid] then set Beg := Mid + 1
5. Else if key = List[Mid] then Loc := Mid,
   print : Loc and exit
6. Set Mid := (Beg + End) / 2
7. If Loc = 0 Writ: Item is not in List
8. Exit

Farhan Hassan

Bynary search
~~Bubble sort~~ (decending)

Example: 190 75 55 30 20 10 6
Beg → ① 2 3 4 5 6 ⑦ → End

1. Set LOC := 0, Beg := 1, End : N and Mid := (Beg+End)/2
2. Repeat steps 3 to 6 while Beg <= END
3. If Key. < L[Mid], then set Beg = Mid+1
4. Else if key > L[Mid] then set End = Mid-1
5. Else if key = List[Mid] the Loc = Mid
   print : Loc and exit.
6. Set Mid = (Beg + End)/2
7. If Loc = 0 Write: Item is not in List
8. Exit

12 August 2026

O ← big O (O) notation
Ω ← big omega
Θ ← big theta

Complexity of Algorithms

```
int main() { printf("Hello world"); }
```

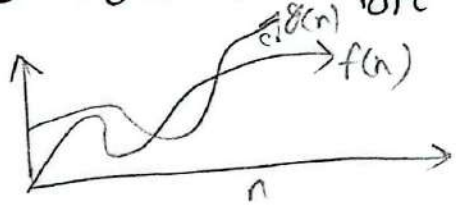
$f(n) = 1$ → constant

```
int main(){
for(int i=0; i<N; i++) printf("Hello");}
```

$f(n) = n$ → linear

Asymptotic Notation

1. O-notation – for the upper bound running time
2. Ω-notation – for the lower bound running time
3. Θ-notation – for the average bound running time



1. O-notation

- A function $f(n) = O(g(n))$ if there are positive constants c and n_0 such that
  $f(n) <= c \cdot g(n)$ for all $n >= n_0$
- when $f(n) = O(g(n))$, it is guarantee that f(n) grows at a rate no faster than g(n)
  So g(n) is an upper bound on f(n)

যার n এর সাপেক্ষে সবচেয়ে বেশি তাকে focus করে বের করি

$f(n) = O(g(n))$, c = positive constant

iff $f(n) \leq c \cdot g(n)$ for all $n_0 \geq n$

(if and only if)

$f(n) = 5n + 50$

$g(n) = n^1$

(সবচেয়ে সাধারণ)

$f(n) \leq c \cdot g(n)$

$5n + 50 \leq c \cdot n$

~~যদি $c = 1$~~

$5n + 50 \leq 1n$ কখনোই true হয় না

$c = 6$ (bare minimum)

$5n + 50 \leq 6n$

$\Rightarrow 50 \leq 6n - 5n$

$\Rightarrow n \geq 50$

n (number of input)

$n_0 = 50$

n এর value 50 বা তার বড় হলে

$f(n) = O(g(n)) = O(n)$

linear

13 August 2026

Slide

~~Lecture 24~~

1. O- notation (upper bound)

Slide ~~31~~ 31, 32.

33

```
Algorithm Linear Search(a, n, k)
{
    for i := 1 to do    // 0 to n-1      | n+1
        if a[i] = k                       | n
            return i;                     | 1/0
    return -1;                            | 0/1
}
```

$n+1+n+1$

Total step count $= 2n+2$ (Worst Case)

$f(n) = 2n+2$

$g(n) = n$

$f(n) \leq c \cdot g(n)$

$\Rightarrow 2n+2 \leq c.n$

$\Rightarrow 2n+2 \leq 3n$

$\therefore n \geq 2 \longrightarrow n_0$

$f(n) = O(g(n)) = O(n)$

Bynary search

1	2	3	4	5	6	7	8
5	6	7	11	15	17	20	21

key = 20

mid

$\frac{n}{2^0}$ $\frac{n}{2^1}$ $\frac{n}{2^2}$ ..... $\frac{n}{2^k}$

$n = 2^k$

$\Rightarrow \log_2 n = k \log_2 2$

$\Rightarrow \log_2 n = k \log_2 2$

$\Rightarrow \log_2 n = k \times 1 = k$

$k = \log_2 8$

$= \log_2 2^3$

$= 3 \log_2 2 = 3 * 1 = 3$

$O(\log_2 n)$

Ans.

19/08/2026

next week বুধবার CT
27 তারিখ @ সিলেবাস আগামিকালের ক্লাস পর্যন্ত

Mid question-1

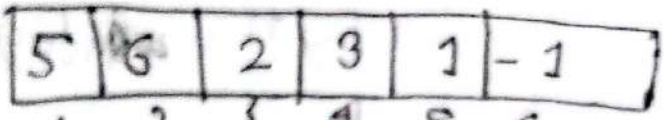
Sorting

(Bubble Sort, Insertion Sort, Selection Sort)

Counting sort এই semester এই হবেনা
heap sort আর radix sort

6 ফ্রাকশন

Bubble sort



j J+1

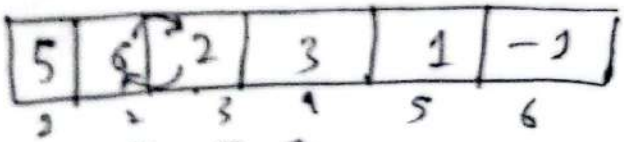
descending হবে ⟨ চিহ্ন এ

```
for(i=1; i<N; i++){
   for(j=1; j<N; j++){
      if(a[J]>a[J+1])
         swap(a[J],a[J+1])
   }
}
```

a[J] < a[J+1)

a[1] a[2]

5 6



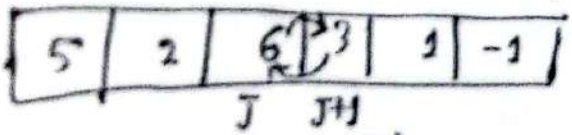
J J+1

a[J] ≥ a[J+1]

a[1] > a[3]

6 ⟨ 2

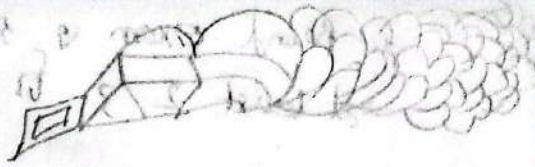
swap(a[J], a[J+1])

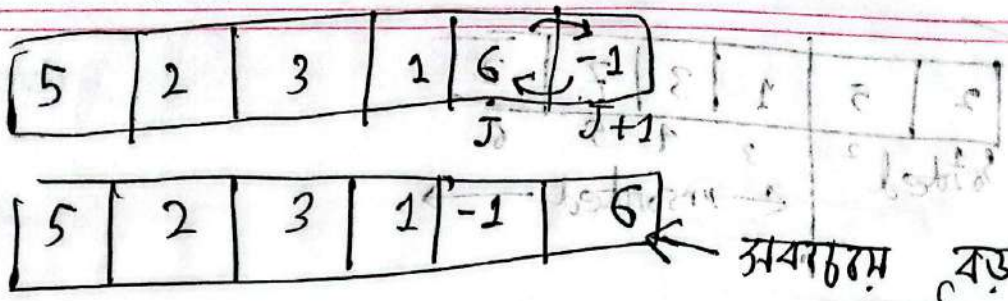


5 2 3 6 1 -1

J J+1

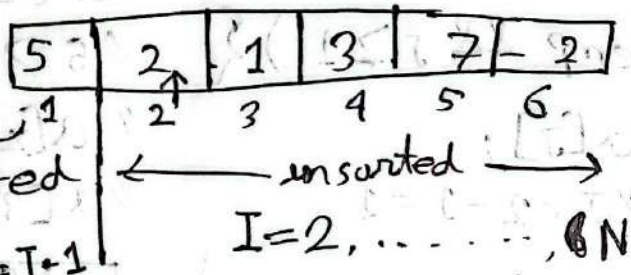
বড় bubble সবচেয়ে আগে যাবে
বড় সংখ্যা ,, ,, sort হয়।
এই নাম bubble sort





সবচেয়ে বড় সংখ্যা
চলে শেষে চলে যাচ্ছে

~~Selection~~ sort
Insetion



Round 1

I = 2
J = 2 - 1 = 1
~~temp = 2 (I এর location বা value)~~
temp = a[I] = a[2] = 2
while (~~temp~~ arr[J] > temp && J >= 1) {

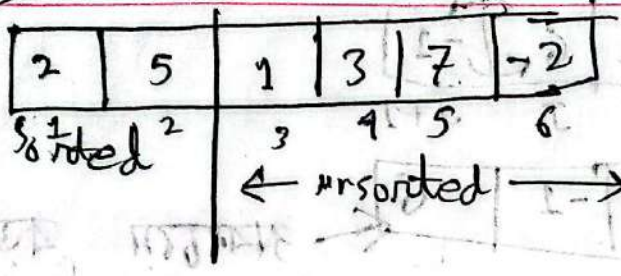
1. a[J+1] = a[J] // 5 5 (1 2)
2. J = J - 1 // J=1 -1 = 0

}
//After while loop
a[J+1] = temp
a[1] = 2

2	5

Round exam এ সব না এলেও
at least 2/3 টা Round নিশ্চয়ই

Round 2



I = 3

J = 3-1 = 2

temp = a[I] = a[3] = 1

while(a[J] > temp && J >= 1){

a[J+1] = a[J];

J = J-1 // J = 2-1 = 1

}

a[J+1] = temp

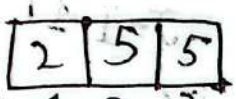
//a[1] = temp

//a[2]

//5 > 1

// a[J+1] = a[2]

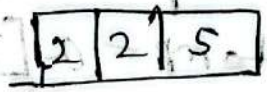
//a[3] = 5

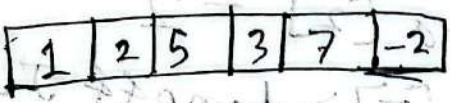


a[1] (2 > 5)

a[1+1]

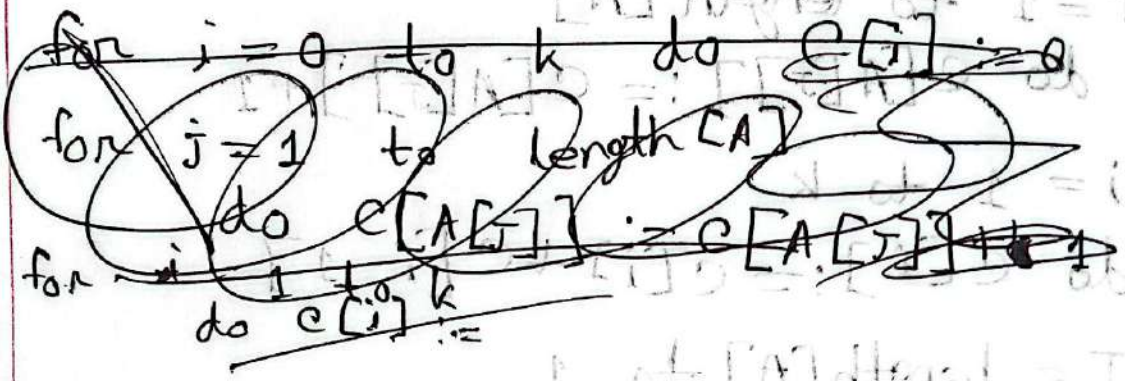
a[2] = 2





20/08/2026

Counting_Sort (A, B, C, k)



A[8] = | 2 | 5 | 3 | 0 | 2 | 3 | 0 | 3 |
1 2 3 4 5 6 7 8

(0, 1, 2, 3, 4, 5)

k = max − min + 1 = 5 − 0 + 1 = 5 + 1 = 6

C[6] = | 0 | 0 | 0 | 0 | 0 | 0 |
0 1 2 3 4 5
i

minimum value 0 থেকে position নিবে

C[i] = 0

C[0] = 0; C[1] = 0 ..... C[5] = 0

C[6] = | 0 | 0 | 0 | 0 | 0 | 0 |
0 1 2 3 4 5

C[A[j]] = C[A[j]] + 1

j = 1

C[A[1]] = C[A[1]] + 1

C[2] = C[2] + 1

= 0 + 1

= 1

min ↓ max ↓

```
for i = 0 to k do C[i] := 0
for J = 1 to length [A]
    do C[A[J]] := C[A[J]] + 1
for i = 1 to k
    do C[i] := C[i] + C[i-1]
for J = length[A] to 1
    do B[C[A[J]]] := A[J]
       C[A[J]] := C[A[J]] - 1
```

ব্যাখ্যা

~~C[A[J]] = C[A[J]] + 1~~

second loop এর পর

C[6] =

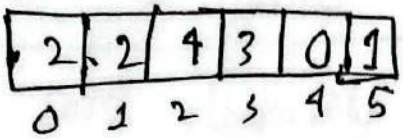
2	0	2	3	0	1
0	1	2	3	4	5

$C[i] = C[i] + C[i-1]$

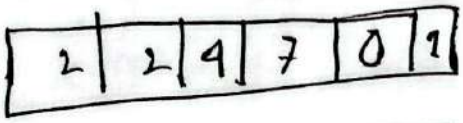
$i=1 \Rightarrow C[1] = C[1] + C[1-1] = 0 + C[0] = 0 + 2 = 2$

2	2	2	3	0	1
0	1	2	3	4	5

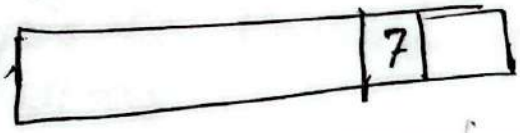
j=2: $C[2] = C[2] + C[1] = 2 + 2 = 4$



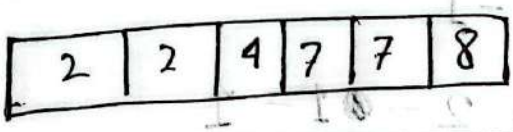
j = 3



j = 4

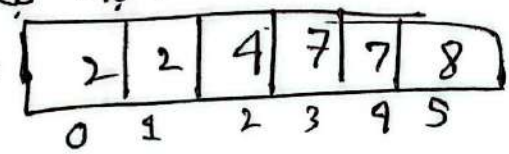


j = 5

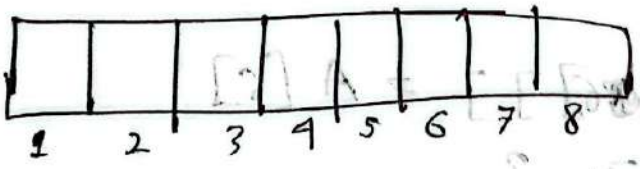


3rd loop শেষ হয়।

$C[6] =$



$B[8] =$



j=8 $B[C[A[j]]] = A[j]$

$\Rightarrow B[C[A[8]]] = A[8]$

$\Rightarrow B[C[3]] = 3$

$\Rightarrow B[7] = 3$

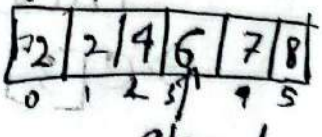
j = 8

$C[A[8]] = C[A[8]] - 1$

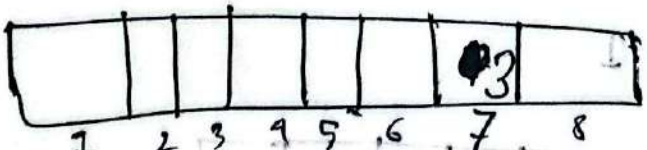
$\Rightarrow C[3] = C[3] - 1$

$= 7 - 1 = 6$

$C[6] =$



$B[8] =$



j=7 $B[c[A[7]]] = A[7]$

$\Rightarrow B[c[0]] = 0$

$\Rightarrow B[2] = 0$

B

1	2	3	4	5	6	7	8
	0					3	

J=7

$c[A[7]] = c[A[7]] - 1$

$c[0] = c[0] - 1 = 2 - 1 = 1$

c

1	2	4	6	7	8
0	1	2	3	4	5

J=6 $B[c[A[6]]] = A[6]$

$\Rightarrow B[c[3]] = 3$

$\Rightarrow B[6] = 3$

B

1	2	3	4	5	6	7	8
	0				3		8

$c[A[6]] = c[A[6]] - 1$

$\Rightarrow c[3] = c[3] - 1$

$= 6 - 1$

$= 5$

c

1	2	4	5	7	8

J এর Value A এর index
A " " C " "
C " " B " "
B " " =A এর Value
C এর Value 1 minus

Simulation করতে বললে Algorithm লেখা নিরর্থক হবে। Algorithm লেখার দরকার নাই

Counting sort stability Maintain করতে slide এর ফুল k=5 করতে হবে
Counting sort Sequance " " "
time complexity কমি ~~তিনটা~~ থেকে যায়

Senerio এর depend করে Sort Algorithm pick করা লাগবে।

CT এ চায় sort
Cselection sort করতে পারি নাই তাই CT তে নাই)

next week এ লাস্ট ক্লাসের CT (ইনশাআল্লাহ) Sorting, searching realated minimum প্রশ্ন থাকবে

2/09/2026

5	2	3	1	7
1	2	3	4	5

Insertion Sort

i

j

min = i = 1

j = i+1 = 1+1 = 2

if(A[min] > A[j])

A[1] > A[2]

5 > 2

min = j = 2

j = i+1 = 2+1 = 3

if(A[min] > A[j])

A[2] > A[3]

2 ≯ 3

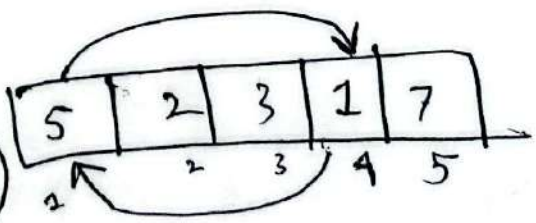
~~min = j~~

~~j = 3+1 = 4~~ (2 থেকে 5 পর্যন্ত)

Loop চলে, এবং সর্বনিম্ন অবস্থান 1 এর index থাকবে min এ

min=4

swap(A[min], A[i])



Radix Sort না পড়লেও Exam এ নাই।

After Round 1

1	2	3	5	7

Next Round শুরুতে Loop চালাব.

আমরা Loop N times

দ্বিতীয় Loop এ প্রায় N times

তাই Worst case এ N×N times

∴ Time complexity $O(N^2)$ (Nested loop এলে সাধারণত $O(N^2)$)

Selection, Insertion, Bubble sort এর $O(N^2)$

Counting Sort

k = max - min + 1

C[k]

A, B → N

C → k

for (i = 1 ---- k) → N times

for (i = 0 ---- k) → k times

k = 100

যদি N = 100

$O(N+k) = O(100+100) = O(200)$

$O(N^2) = O(10000)$

যদি N = 1000

$O(N+k) = O(1100)$

$O(N^2) = O(1000000)$

Time complexity $O(N+k)$

★ ADT কী

Stack

Abstract data type

ADT কী

Abstract — নিজের structure নাই।
different data type কে help করার

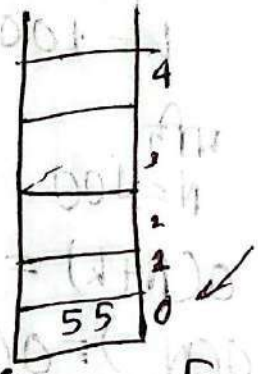
LIFO — Last In First Out

Stack LIFO concept মেনে করে।

Special Variable **TOP**

primary operator দুইটি ~~Push → Pop~~

push — insert
pop — delete



Algorithm Based ও 1 থেকে শুরু
কিন্তু সুবিধার্থে এখানে 0

Size = 5

Top = -1

Push (value)

Top++ → 0

Stack [Top] = value
0 55

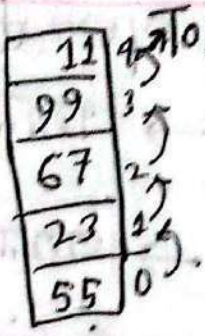


Push 23

Top ++ → 1

stack [TOP] = 23

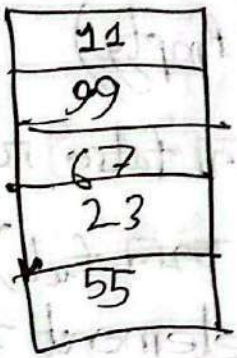
1



Top == Size - 1

→ Push করলে overflow

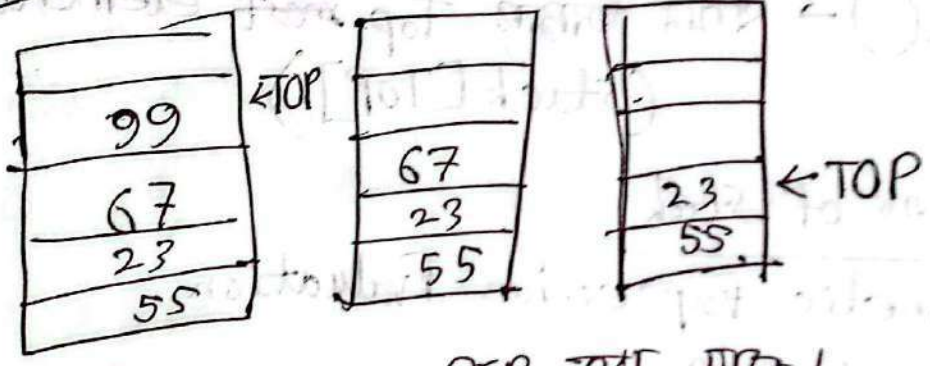
POP() Example



TOP আছে যে

stack [TOP] = NULL

Top --



5 টা ডিস্ক। 5 বার POP করা যাবে।



If (Top = -1) Underflow

Stack Example: Text editor এর undo, ~~redo~~

Secondary operations — (Index)

1. TOP() → এখন TOP কোথায় আছে তা return করে
2. size() → এখন আমার size কত (size = Top+1) (কয়টি সংখ্যা আছে)
3. isEmpty() → Empty হলে True নাহলে false return (যখন Top = -1 তখন Empty)
4. isFull() → Full হলে True নাহলে false return (যখন Top = size-1 তখন full)
5. peek() → এখন আমার Top. ~~not~~ element কত (stack[Top])

Applications of Stack

- Arithmetic Expression Evaluation
- Syntax Parsing
- Solving search Problem

Arithmetic Expression

Three types of arithmetic expression:

1: Infix Expression

2 + 4, a − b/c

2: Prefix expression

+ 2 4, − a/b c

3: Postfix Expression

2 4 +, a b c / −

Infix → Prefix

2. F = (5 + 7) * 8 − 10

step 1: 12

step 2: 12 * 8 − 10
96

step 3: 86

Infix এ 3 বার Round ঘুরা লাগলে Computer এ অনেক বড় Problem

~~Parenthesis balanced~~ ~~কিনা~~ ~~চেক~~ ~~করার~~ ~~Lab task~~ ~~ছিল~~

1 (carret)

Infix → Postfix

5 * (6+2) - 12/4 (প্রথমে আমরা

Infix	stack	Postfix
5		5
*	* (0 1 2 3 4)	5
(	* ((0 1)	5
6	* (	5,6
+	* (+ (0 1 2)	5,6
2	* (+ (0 1 2)	5,6,2
)	* (0 1)	5,6,2,+
-	-	5,6,2,+,*
12	-	5,6,2,+,*,12
/	- /	
4	- /	5,6,2,+,*,12,4

Empty stack এ আসার problem হয়

stack এ Top এ push

1 * / + ^

25 নং slide next দিন নিয়ে বসতে দিবে। 1 মার্কস Free

আসার পর first এই push

+ কে pop করে postfix এ নামাবে (কে pop করে চলে যাবে

5,6,2,+*

same priority তে আসলে চিন্তা করে চলে যাবে

stack খালি না। তবে এর চেয়ে বড় priority আসবে। * এর priority বেশী * চলে যাবে (pop) - আসবে।

5,6,2,+,

5,6,2,+*

^ বড় Priority আসলে pop হবে push

expression শেষ stack 1 by 1 pop

5,6,2,+,*,12,4,/,-

- এর চেয়ে Powerful

1. 8 * (5 ^ 4 + 2) - 6 ^ 2 / (9 * 3)

Infix expression	stack	Postfix Expression
8		8
*	*	8
(	* (	8
5	* (	8 5
^	* (^	8 5
4	* (^	8 5 4
+	* (+	8 5 4 ^
2	* (+	8 5 4 ^ 2
)	*	8 5 4 ^ 2 +
-	-	8 5 4 ^ 2 + *
6	-	8 5 4 ^ 2 + * 6
^	- ^	8 5 4 ^ 2 + * 6
2	- ^	8 5 4 ^ 2 + * 6 2
/	- /	8 5 4 ^ 2 + * 6 2 ^
(	- / (	8 5 4 ^ 2 + * 6 2 ^
9	- / (	8 5 4 ^ 2 + * 6 2 ^ 9
*	- / (*	8 5 4 ^ 2 + * 6 2 ^ 9
3	- / (*	8 5 4 ^ 2 + * 6 2 ^ 9 3
)	- /	8 5 4 ^ 2 + * 6 2 ^ 9 3 *
		8 5 4 ^ 2 + * 6 2 ^ 9 3 * / -

2. A + (B * C - (D / E ^ F) * G) * H

Infix expression	stack	Postfix expression
A		A
+	+	A
(	+ (	A
B	+ (	A B
*	+ (*	A B
C	+ (*	A B C
-	+ (-	A B C *
(	+ (- (	A B C *
D	+ (- (	A B C * D
/	+ (- (/	A B C * D
E	+ (- (/	A B C * D E
^	+ (- (/ ^	A B C * D E
F	+ (- (/ ^	A B C * D E F
)	+ (-	A B C * D E F ^ /
*	+ (- *	A B C * D E F ^ /
G	+ (- *	A B C * D E F ^ / G
)	+	A B C * D E F ^ / G * -
*	+ *	A B C * D E F ^ / G * -
H	+ *	A B C * D E F ^ / G * - H
		A B C * D E F ^ / G * - H * +

Post-fix Calculation ৭ প্রতি Iteration এ হয় যায়

3/9/26 Post-fix to value

Example: 5, 6, 2, +, *, 12, 4, /, -

Postfix expression	Stack
5	5
6	5 6
2	5 6 2
+	5 8
*	40
12	40 12
4	40, 12 4
/	40 3
-	37

Sign পাওয়া মাত্র pop হয়ে যায়

Pop এর সময়
A = 2
B = 6
B ϕ A
6 + 2
= 8

Phi (ϕ) দ্বারা operation

(-) বা (/) এর সময় মাঝে গড়বিল না হয়। তাই B ϕ A হয়।

operators	precedence
( )	Highest
^	Next highest
*, /, %	Next ^ highest
+, -	Lowest

^
/*
+ -

1. Evaluate

8 5 4 ^ 2 + * 6 2 ^ 9 3 * / -

Prefix expression	Stanck
8	8
5	8 5
4	8 5 4
^	8 625
2	8 625 2
+	8,627
*	5016
6	5016 6
2	5016 6 2
^	5016 36
9	5016 36 9
*	5016 324
/	15.18
3	5016 36 9 3
*	5016 36 27
/	5016 1.33333333
-	5014.666666666666666667

9/9/25

Linear queue এর problem
যেটা Queue সমস্যা bad overflow

Queue

ADT

Array/LinkedList দিয়ে করা যায়

Linear queue

1. Insert (Enqueue)
2. Delete (Dequeue)

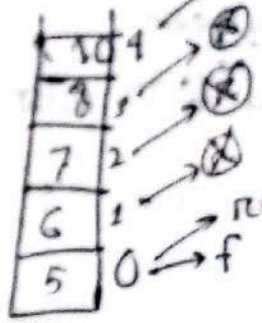
FIFO list



Stack[5]

Top = -1

Push() → Top++
stack[Top] = value



Queue[5]
rear = -1
front = -1

Enqueue
rear = N-1 → overflow
Dequeue
front > rear → underflow

(insert)
(delete)

Insert (Enqueue)

if (front == -1 && rear == -1)
front = 0;
rear++
queue[rear] = value

Stack এর Queue এর Insert almost same

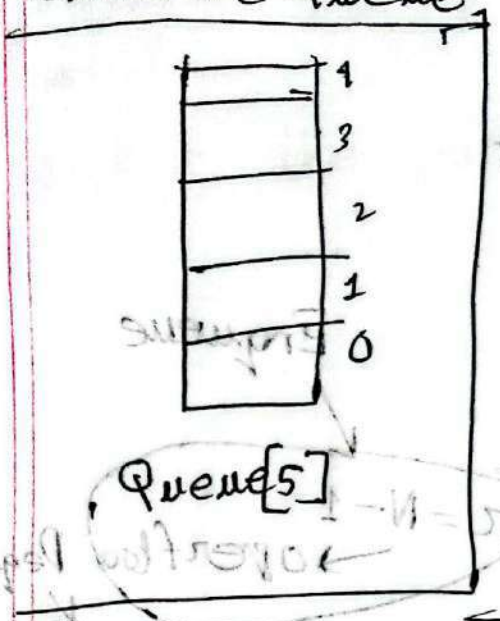
Dequeue (Delete)

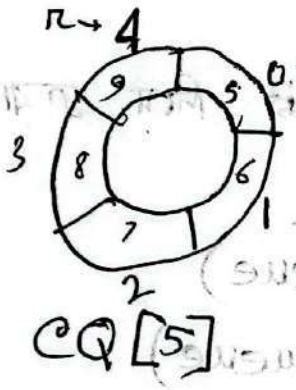
front++ (condition উল্টে আসবে)

"Reuse" নিয়ে question আসবে

random access ও support করবে, Insertion & deletion can be done from different ends (Queue)

Circular Queue





Enqueue

Condition 1: ~~if (~~

if (f = r = -1) → f = 0

Condion 2:

if ((r == N-1 && f == 0) || r+1 == f)

→ overflow

Contion 3:

if (r == N-1)

→ r = 0

Condition 4: CQ[r] = value

r++

CQ[r] = value

Dequeue ()

Condition 1:

if(f > r)
↳ underflow

Condition 2:

if (f = r = N-1)
f = 0

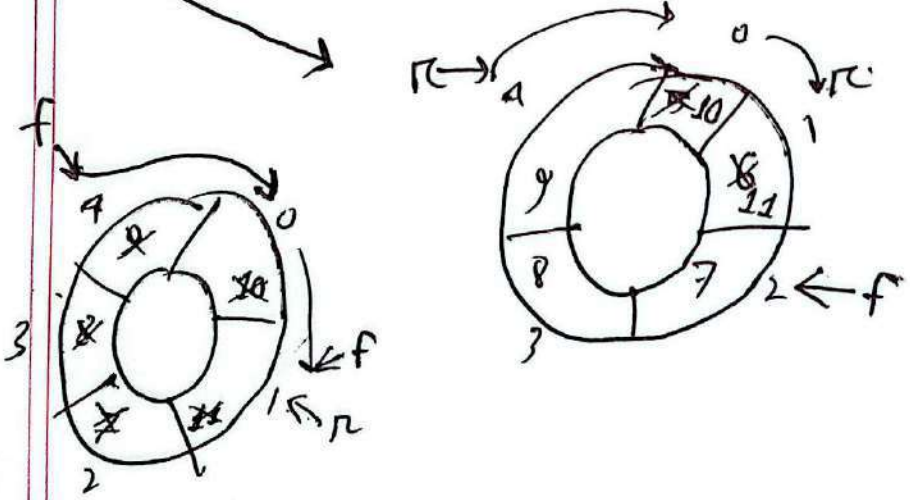
Condition 3:

if(r == f)
f = -1
r = -1

optional
(যেন Queue খালি তাই reset করা।
না দিলেও মার্ক করতে কোন সমস্যা নেই।

Condition 4:

CQ[f] = NULL
f++



এইবারের Mid এ দুটো স্পেশালি প্রশ্ন করবে।
most recent. term টা মানেই LIFO (stack) হবে।

Fall 25 Mid question এ আসবে।



Department of Computer Science & Engineering
University of Asia Pacific (UAP)

Class Test #1 Spring 2026 2nd Year 1st Semester

Course Code: CSE 203 Course Title: Data Structures and Algorithms I Credits: 3

Full Marks: 10 Duration: 30 minutes

Instructions:

1. Part marks are shown in the margins.
2. Non-programmable calculators are allowed.

Name:	ID:

1. Linear search normally examines elements sequentially until the target is found. Suppose an unsorted array contains duplicate values, and you need to determine the first occurrence, last occurrence, and total number of occurrences of a given element. [4]

   Explain how linear search can be modified to accomplish all three tasks in a single traversal of the array.

2. Below is the pseudocode for counting sort. There are some errors in it.

```
Counting_Sort( A, B, C, k, N)
 i.   for j = 1 to N
ii.       do C[A[j]] := C[A[j]]+1
.ii.  for i = 1 to N
iv.       do C[i] := C[i]+ C[i-1]
 v.   for j = N to 1
vi.       do C[A[j]] := C[A[j]]-1
'ii.      A[C[B[j]]] := A[j]
.ii.  End.
```

   a. Find the errors. Explain why and how the above-stated code would produce wrong results using one example. [3]

   b. Now correct the pseudocode. [3]

CamScanner

pseudocode

1. Linear search ~~algorithm~~ is given below which can accomplish all three tasks-

Linear_Search (Array, N, key, first, last, count)

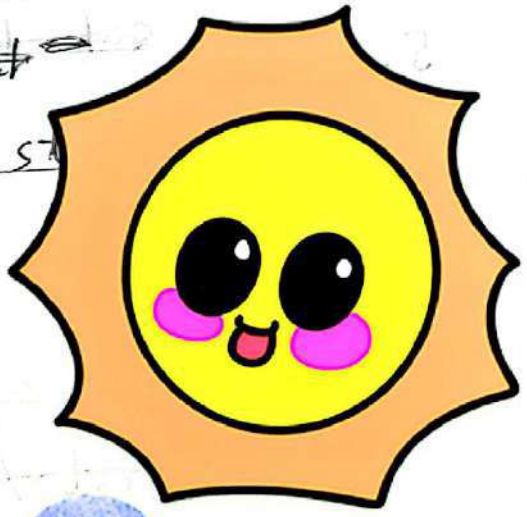
1. set first := -1, last := -1 and count := 0
2. for i = 1 to N do:
3. If Array[i] = key then
4. ~~set first = i~~
   If first = -1 then
5. set first := i
6. set last := i
7. Set count := count + 1
   ~~If first ≠ -1 then Print the value of~~
   ~~If first ≠~~
8. If count > 0 then
9. print "First occurance Position: ", first
10. print "Last occurance Position: ", last
11. print "Total occurance: ", count
12. Else
13. print "Element not found in array
14. End

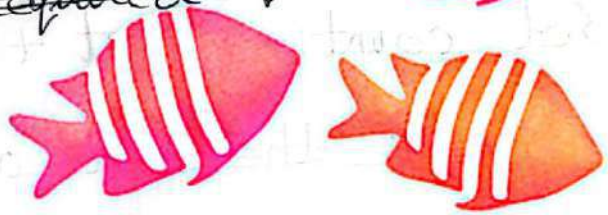
Here three variables (first, last, count) are used to perform the three task. At the begenning first and last are set to -1 and count is set to 0. While loping throug array It is

the element matches the ~~value~~ key that need to be searched. If Its matched the first is updated ~~to~~ as that position if first is -1. If first is not -1 the it was already updated before so it only updates once. The last is updated whenever key is matched so after the last occurance it updates for the last time. count is incremented whenever the key matched. ~~Does th~~

~~us these three va~~ ~~st~~
~~required~~.









আমি ছোট করে লিখতে পারিনা (৩ পৃষ্ঠা) 😊

2(a) There are four errors-

1. Initialization missing:

Array C was suppose to be set to 0 at the begenning. But the loop isn't there. As a result C will hold garbage value that ~~will give wrong ans~~ can cause out-of-bound error.

2. Incorrect loop limit in line (v)

The third loop was supposed to loop from 1 to K which is looping ~~N-1~~ from 1 to N.

3. At line (vi) C[A[J]] is decremented before placing ~~the~~ element in the output array. Line (vi) and (vii) need to be exchanged

4. At line (vii) ~~A~~ A array is updated instead of array B.

Example:

Let an array a[5] = {2, 5, 4, 1, 0} here k = 5-0+1 = 6

Since B isn't intialized it will hold garbage value. For simplicity lets assume ~~c~~ with garbage values ~~b~~ ~~c~~[k] = ~~c~~[6] = {5, 10, 0, 3, 5, 2}

~~After fi~~ ~~Array~~ C will ~~be~~

In the first loop

$j = 1 \rightarrow c[A[1]] = c[A[1]] + 1$

$\Rightarrow c[2] = c[2] + 1 = 0 + 1 = 1$

	0	1	2	3	4	5
C =	5	10	10	3	5	2

$J = 2 \rightarrow c[5] = c[5] + 1 = 2 + 1 = 3$

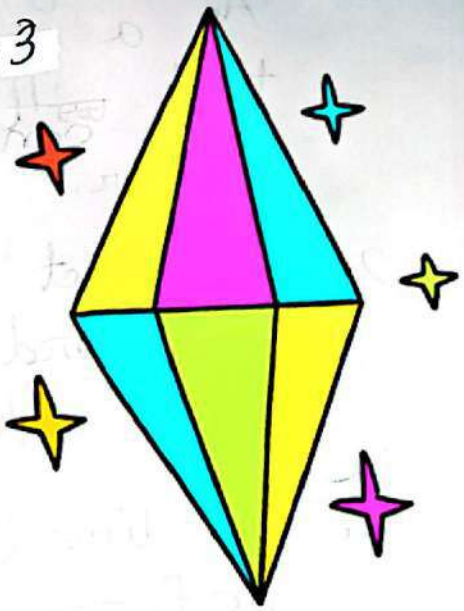
C =	5	10	01	3	6	3

Similarly -

$J = 3 \rightarrow$ C = | 5 | 10 | 01 | 3 | 6 | 3 |

$J = 4 \rightarrow$ C = | 5 | 11 | 01 | 3 | 6 | 3 |

$J = 5 \rightarrow$ C = | 6 | 11 | 1 | 3 | 6 | 3 |



In the second loop -

$i = 1 \rightarrow C[1] = C[1] + C[0] = 11 + 6 = 17$ ~~$10 + 5 = 15$~~

C = | 6 | 17 | 1 | 3 | 6 | 3 |

similary

$i = 2 \rightarrow$ C | 6 | 17 | 18 | 3 | 6 | 3 |

$i = 3 \rightarrow$ C = | 6 | 17 | 18 | 21 | 6 | 3 |

$i = 4 \rightarrow$ C = | 6 | 17 | 18 | 21 | 27 | 3 |

$i = 5 \rightarrow$ C = | 6 | 17 | 18 | 21 | 27 | 30 |



In the third loop

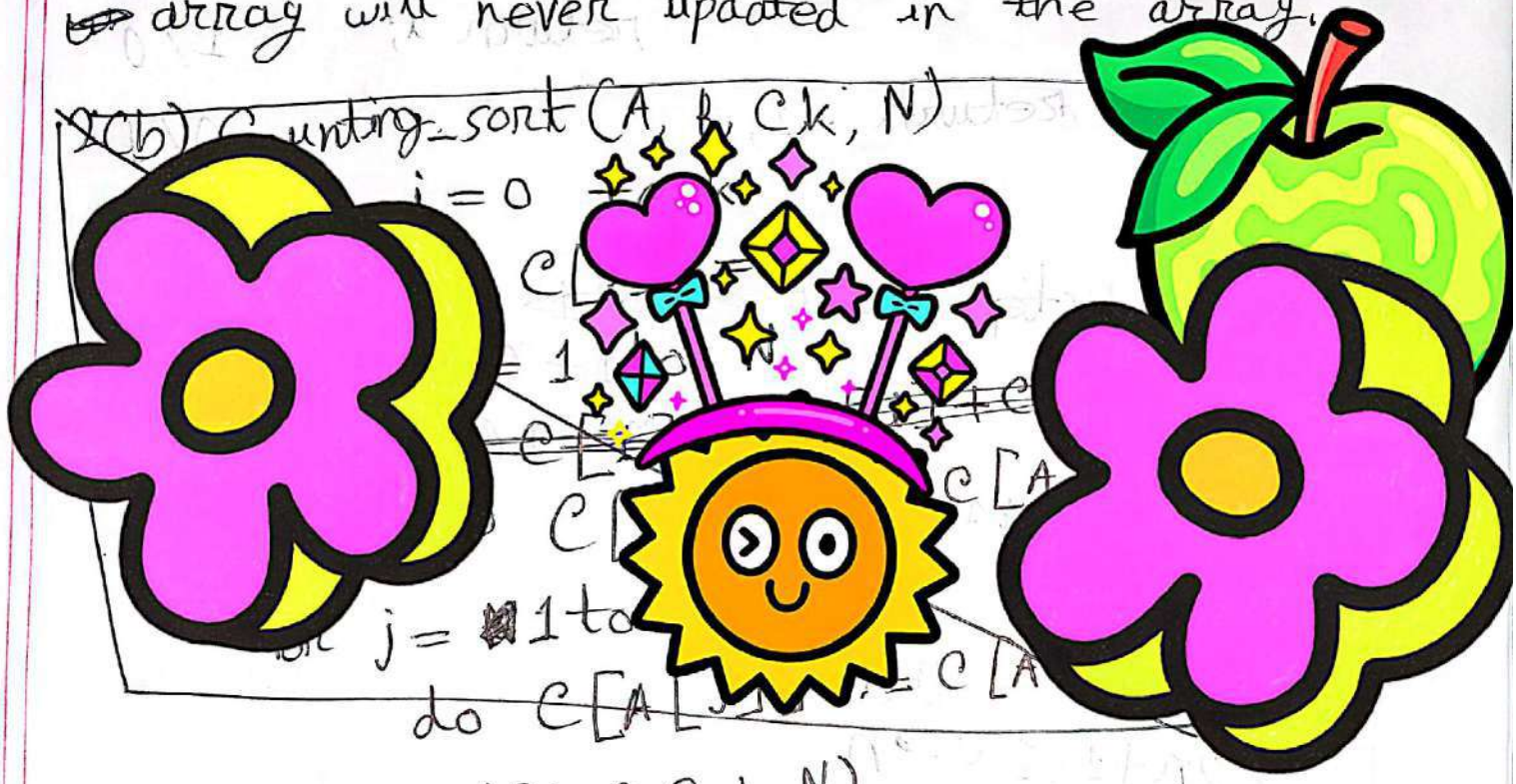
$J = 5 \rightarrow C[A[5]] = C[A[5]] - 1$

$\Rightarrow C[0] = C[0] - 1 = 6 - 1 = 5$

C | 5 | 17 | 18 | 21 | 27 | 30 |

and $A[C[B[5]]] = A[5]$

Here A is updated and B holds a garbage value. B is the array which is suppose to hold the sorted array. Thus the requared sorted array will never updated in the array.



2(b) Counting_Sort(A, B, C, k, N)

```
for i = 0 to k
    do C[i] := 0
for j = 1 to N
    do C[A[j]] := C[A[j]] + 1
for i = 1 to k
    do C[i] := C[i] + C[i-1]
for j = N to 1
    do B[C[A[j]]] := A[j]
    C[A[j]] := C[A[j]] - 1
End
```

Department of Computer Science & Engineering
University of Asia Pacific (UAP)

Class Test #1 **Spring 2026** **2nd Year 1st Semester**

Course Code: CSE 203 **Course Title: Data Structures and Algorithms I** **Credits: 3.0**

Full Marks: 10 **Duration: 30 minutes**

Instructions:

Part marks are shown in the margins.

Non-programmable calculators are allowed

Name: **ID:**

1. Why is Binary Search more efficient than Linear Search for large inputs? Derive the time complexities for both algorithms and then prove your case. [4]

2. Below is the algorithm for binary search. There are some errors in it.

Algorithm: Binary-Search (L, N, Key)

1. Set Loc := -1, Beg := 0, End :=N
2. Repeat steps 3 to 6 while Beg <= End
3. Mid := (Beg + End)/2
4. If Key < L[Mid], then Set End := Mid
5. Else if Key>L[Mid] then Set Beg := Mid +1
6. Else if Key = Mid then Loc:= Key, print: Loc.
7. If Loc = 0 Write: Item is not in List.
8. Exit.

i. *Find the errors. Take 5 random integers that are sorted in ascending order and a key value of your choice. Explain why and how the above-stated code would produce wrong results using your example.* [4]

ii. Now correct the pseudocode.

CamScanner

1. Linear_Search(a, n, k)

```
{
    for i := 1 to n do          n+1
        If a[i] = k             n
            return i;           1/0
    return -1;                  0/1
}
```

Total step count
$= n+1+n+1+0$
$= 2n+1$ (Worst Case)

$f(n) = 2n+1$
$g(n) = n$

Now, $f(n) \leq c \cdot g(n)$
$\Rightarrow 2n+1 \leq c \cdot n$

The inquity cant be true unless c is at least 3

$\therefore 2n+1 \leq 3n$
$\Rightarrow 3n-2n \leq 1$
$\Rightarrow n \leq -2$
$\Rightarrow n \geq 2$

$\therefore$ if n is greater than 2 than
$f(n) = O(g(n)) = O(n)$

In binary search it ~~skips half~~ decreases half of the remaing search space in every step until it reaches 1.

∴ The search space after every step

$$\frac{n}{2^0}, \frac{n}{2^1}, \frac{n}{2^2}, \frac{n}{2^3}, \ldots\ldots, \frac{n}{2^k}$$

here k is the ~~total~~ number of steps







here

$$\frac{n}{2^k} = 1$$

$$\Rightarrow n = 2^k$$

$$\Rightarrow \log_2 n = \log_2 2^k$$

$$\Rightarrow \log_2 n = k(\log_2 2)$$

$$\Rightarrow \log_2 n = k$$

∴ Time complexity is $O(\log_2 n)$

For a large dataset of size $n = 1000000$

linear search requires $n = 1000000$ comparisons

For binary search it requires $\log_2 1000000 = 20$ comparis

here $20 < 1000000$

Therefore Binary search is more efficient than linear search for large inputs.

CamScanner

2. (i) Errors:-

1. At line ① ~~Location~~ Beg need to set as 1 instead of 0. Loc is set to -1, which should be set to 0. ~~since at line~~ Since it compared with 0 at the end.
2. At line 4 End need to set as Mid - 1. Setting it to Mid can cause infinite loop.
3. In line 6 Key is compared with Mid. Mid holds index. So Key ~~should~~ ~~co~~ need to compare with L[Mid]
4. In line 6 Loc is set to Key instead of Mid

Example

L[5] = {10, 20, 30, 40, 50}, Let key = 10

Loc = -1, Beg = 0, End = N = 5

Iteration 1 -

$Mid = \frac{0+5}{2} = 2$

L[Mid] = L[2] = 20

10 < 20 is True. Therefore End = Mid = 2

Iteration 2 -

$Mid = \frac{0+2}{2} = 1$

L[Mid] = L[1] = 10

~~10 = 10 is true. Therefore~~ 10 is not greater or less than 10

So At line 6 it compares 10 = 1 which is false

~~So no~~

Iteration 3 –

$$Mid = \frac{0+1}{2} = 0$$

Here L[Mid] = L[0] is out-of-bound

2(ii)

1. Set Loc := 0, Beg := 1, End := N
2. Repeat steps 3 to 6 while Beg <= End
3. Mid := (Beg + End)/2
4. If Key < L[Mid], then set End := Mid - 1
5. Else if Key > L[Mid], then Set Beg := Mid + 1
6. Else if (Key = L[Mid]), then set Loc := Key, print: Loc.
7. If Loc = 0 Write: Item is not in List
8. Exit







CamScanner